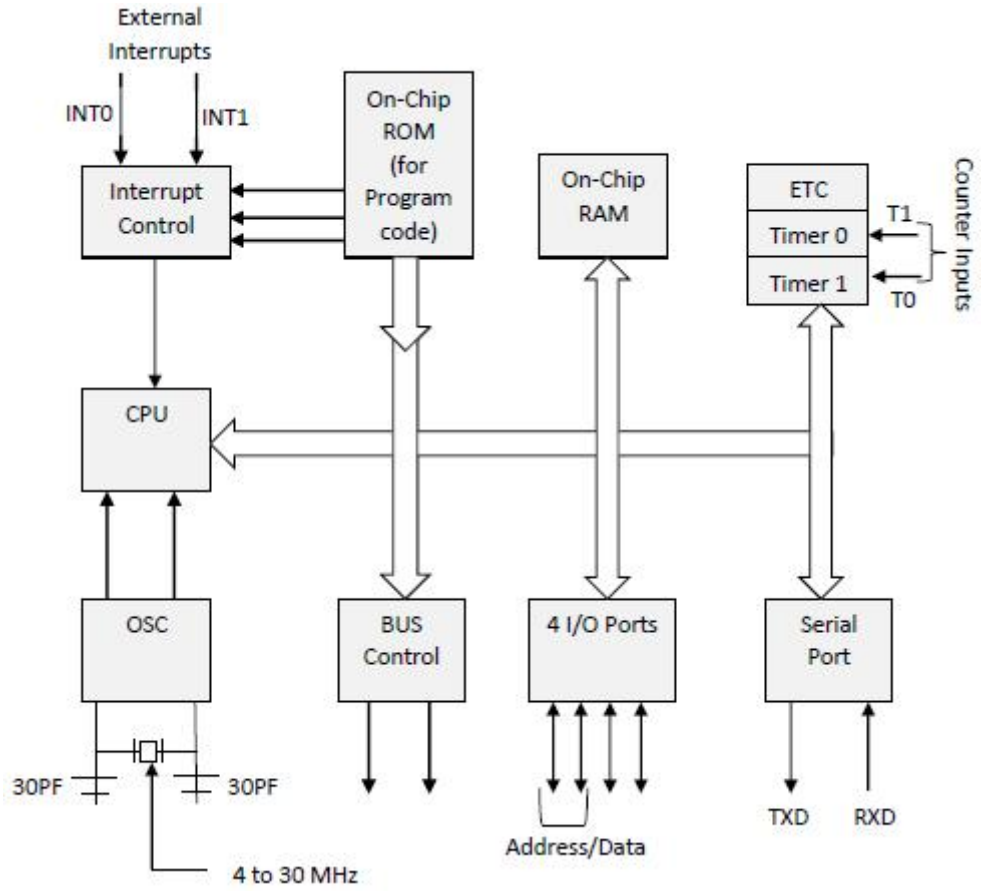
Embedded System and IOT

1.Illustrate the 8051-microcontroller architecture with necessary diagrams



Introduction

The **8051 microcontroller** is an **8-bit embedded processor** introduced by Intel.
It is called a **"System on Chip"** because it integrates CPU, memory, I/O ports, timers, serial communication, and interrupt control on a single chip.

Block Diagram Description of 8051 Architecture

The architecture of 8051 consists of the following major functional blocks:

---

1. Central Processing Unit (CPU)

- The CPU is the **heart of the 8051**
- Performs arithmetic, logic, and control operations
- Includes:
    - **Arithmetic Logic Unit (ALU)**
    - **Accumulator (A)**
    - **B Register**
    - **Program Counter (PC)**
    - **Instruction Decoder**

Functions:

- Fetches instructions from program memory
- Decodes and executes instructions
- Controls data flow within the microcontroller

---

2. Arithmetic Logic Unit (ALU)

- Performs:
    - Addition, subtraction
    - AND, OR, XOR operations
    - Comparisons and bit manipulations
- Works mainly with **Accumulator (A)** and **B register**

---

3. Registers

a) Accumulator (A)

- 8-bit register
- Used for arithmetic and logic operations

b) B Register

- Used for multiplication and division
- Temporary data storage

c) Program Counter (PC)

- 16-bit register
- Holds the address of the next instruction

d) Stack Pointer (SP)

- 8-bit register
- Points to the top of the stack in RAM

e) Program Status Word (PSW)

- Contains flags:
  - Carry (CY)
  - Auxiliary Carry (AC)
  - Overflow (OV)
  - Parity (P)
- Used for condition checking and register bank selection

---

4. Memory Organization

a) Program Memory (ROM)

- On-chip ROM: **4 KB**
- Stores program instructions
- Can be expanded externally up to **64 KB**

b) Data Memory (RAM)

- On-chip RAM: **128 bytes**
- Used for:
  - Register banks
  - Bit-addressable area
  - General purpose RAM

---

5. Input / Output Ports

- **Four 8-bit I/O ports**: P0, P1, P2, P3
- Total **32 I/O pins**

Port Functions:

- **Port 0**: Multiplexed address/data bus
- **Port 1**: General purpose I/O
- **Port 2**: Higher-order address bus
- **Port 3**: Special functions (interrupts, timers, serial I/O)

---

6. Timers / Counters

- Two 16-bit timers:
  - **Timer 0**
  - **Timer 1**

- Used for:
  - Delay generation
  - Event counting
  - Baud rate generation

---

7. Serial Communication Unit

- Built-in **UART**
- Supports full-duplex serial communication
- Uses:
  - **TXD (P3.1)** – Transmit
  - **RXD (P3.0)** – Receive
- Controlled by **SCON and SBUF registers**

---

8. Interrupt Control System

- Supports **5 interrupts**:
  - 2 external
  - 3 internal
- Controlled by:
  - **IE (Interrupt Enable)**
  - **IP (Interrupt Priority)**
  - **TCON register**
- Enables real-time event handling

---

9. Oscillator and Clock Circuit

- On-chip oscillator
- External crystal (typically **12 MHz**)
- Controls instruction execution speed

---

10. Control and Special Function Registers (SFRs)

- Located in upper RAM area
- Used to control:
  - Timers
  - Serial port
  - Interrupts
  - I/O ports

Overall Working of 8051 Architecture

1. Program stored in ROM
2. PC fetches instruction
3. Instruction decoded by CPU
4. ALU executes operation
5. Results stored in registers or memory
6. I/O, timers, serial port, and interrupts operate in parallel

---

Advantages of 8051 Architecture

- Compact and low cost
- Easy to program
- Reliable for embedded systems
- Ideal for real-time applications

Applications

- Industrial control systems
- Home automation
- Traffic light control
- Robotics
- Embedded IoT devices

---

2. Traffic light controller

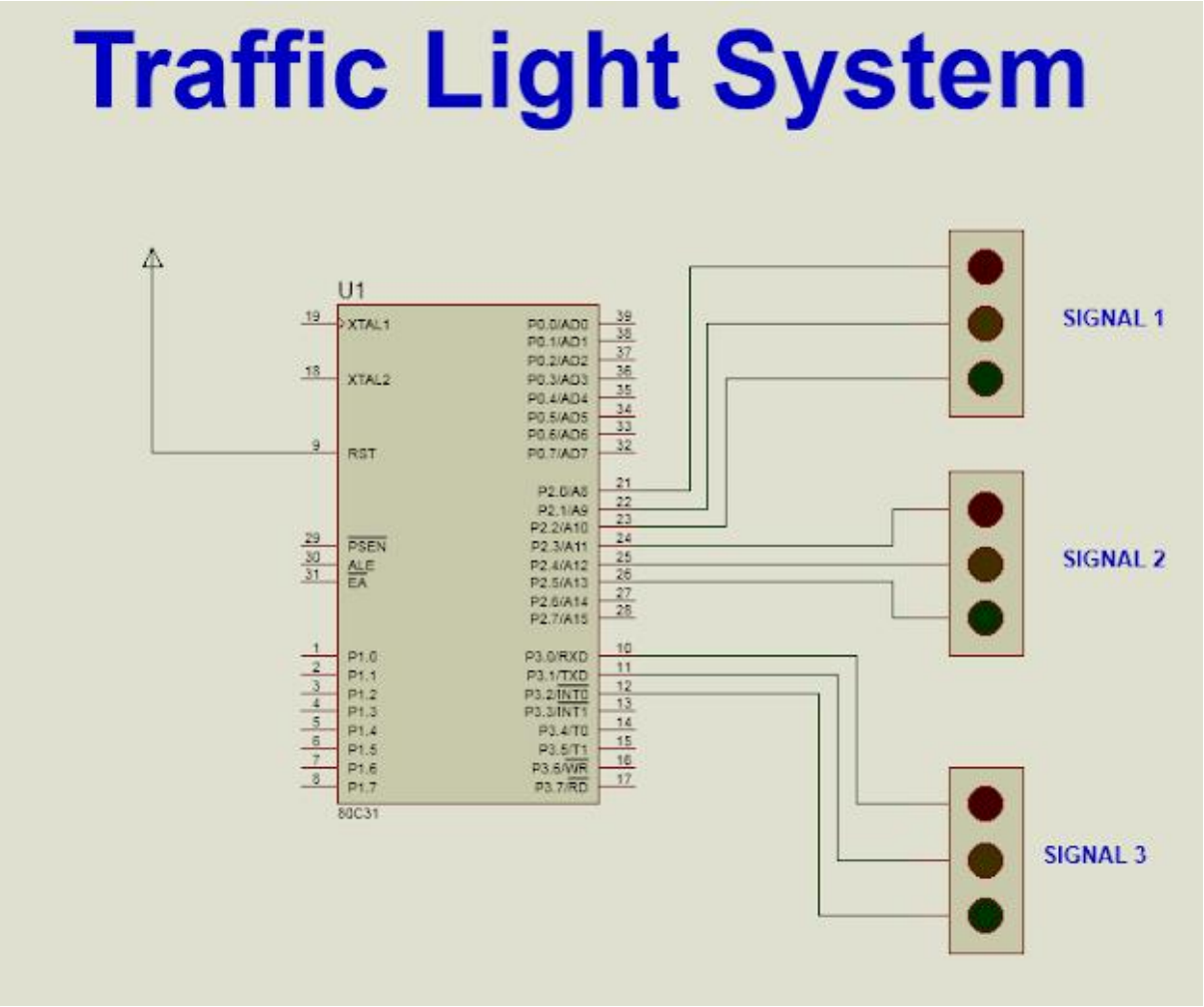
Introduction

A **Traffic Light Controller** is an embedded system that controls traffic signals at road intersections to ensure safe and orderly movement of vehicles.
The **8051 microcontroller** is widely used to implement traffic light control due to its simplicity, built-in timers, I/O ports, and reliability.

Objective

- To control **Red, Yellow, and Green** lights automatically
- To provide **fixed time delays** for each signal

- To ensure **safe traffic flow** at junctions



Components Used

- **8051 Microcontroller**
- **LEDs** (Red, Yellow, Green)
- **Resistors**
- **Power Supply**
- **Crystal Oscillator**
- **Connecting Wires**

Block Diagram Description

- The **8051 microcontroller** acts as the control unit
- LEDs are connected to **I/O ports** of the 8051
- Timers generate required **time delays**
- Output signals drive the LEDs to switch ON/OFF in sequence

Working Principle

The traffic light controller works based on **time-based switching**:

1. **Red light ON** → Vehicles must stop
2. **Green light ON** → Vehicles can move
3. **Yellow light ON** → Indicates transition (Get ready / Slow down)
4. The cycle repeats continuously

Each light remains ON for a **predefined time** using internal timers or delay loops.

Signal Timing Sequence

Signal	Action	Time Duration
Red	Stop	Long delay
Green	Go	Long delay
Yellow	Ready / Slow	Short delay

Assumptions

Port 1 **is connected to LEDs**

- **P1.0 → Red LED**
- **P1.1 → Yellow LED**
- **P1.2 → Green LED**
- Common cathode LEDs are used
- Delay is created using **software delay loop**

8051 PROGRAM (ASSEMBLY LANGUAGE)

```
ORG 0000H          ; Start of program

MAIN:
    MOV P1, #01H   ; RED ON, YELLOW & GREEN OFF
    ACALL DELAY

    MOV P1, #04H   ; GREEN ON, RED & YELLOW OFF
    ACALL DELAY

    MOV P1, #02H   ; YELLOW ON, RED & GREEN OFF
    ACALL DELAY

    SJMP MAIN      ; Repeat the sequence

; ----------------------------
```

```
; Delay Subroutine
; ----------------------------
DELAY:
    MOV R0, #255
D1: MOV R1, #255
D2: DJNZ R1, D2
    DJNZ R0, D1
    RET

END
```

OUTPUT EXPLANATION

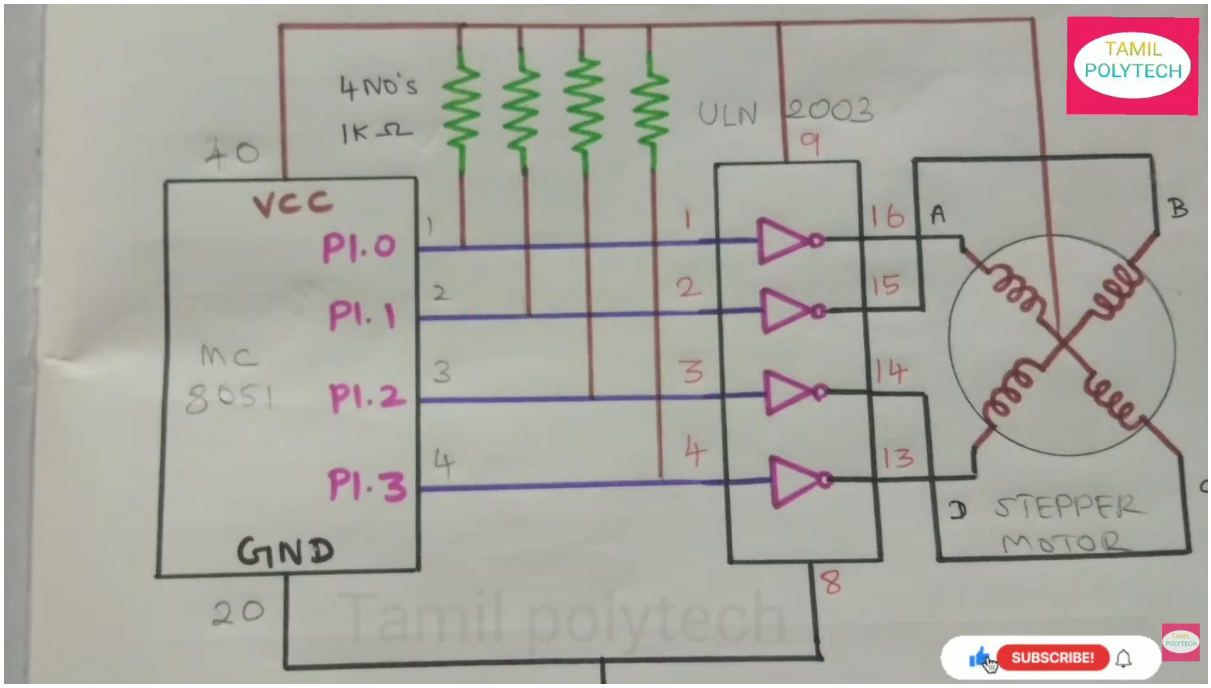
LED Status on Port 1

P1 Value	LED ON	Meaning
01H	Red	STOP
04H	Green	GO
02H	Yellow	READY / WAIT

OUTPUT SEQUENCE (WHAT YOU SEE)

1. 🔴 **Red LED ON** → Vehicles must **STOP**
2. 🟢 **Green LED ON** → Vehicles can **GO**
3. 🟡 **Yellow LED ON** → Vehicles should **WAIT**
4. Sequence **repeats continuously**

3. Describe the operation of stepper motor. Draw the schematic for interfacing a stepper motor with 8051 microcontroller and write a programming for controlling speed and direction of motor




```
+---------+    +-----------+    +--------------+
| 8051    | --> | ULN2003   | --> | Stepper Motor|
|         |    | Driver IC |    |              |
| P1.0    |    |           |    | Coil A       |
| P1.1    |    |           |    | Coil B       |
| P1.2    |    |           |    | Coil C       |
| P1.3    |    |           |    | Coil D       |
+---------+    +-----------+    +--------------+
```

Operation of Stepper Motor

A **stepper motor** is an electromechanical device that converts **digital pulses into precise mechanical rotation**.
Unlike DC motors, a stepper motor rotates in **fixed angular steps**, making it suitable for **position and speed control**.

Working Principle

- The motor consists of **multiple coils (windings)**
- When coils are energized in a **specific sequence**, the rotor rotates step-by-step
- Each pulse from the microcontroller causes **one step movement**
- **Speed** depends on pulse frequency
- **Direction** depends on the order of excitation

Types of Stepper Motors

- Variable Reluctance Stepper Motor
- Permanent Magnet Stepper Motor
- Hybrid Stepper Motor (most commonly used)

Advantages

- Precise position control
- No feedback system required
- Easy digital control

(b) Interfacing Stepper Motor with 8051 Microcontroller

Need for Driver Circuit

8051 cannot directly drive a stepper motor due to:

- Low current capability
- High power requirement of motor coils

Hence, a **ULN2003 / ULN2803 driver IC** is used.

Pin Connections (Example)

8051 Port	Driver IC	Stepper Motor Coil
P1.0	IN1	Coil A
P1.1	IN2	Coil B
P1.2	IN3	Coil C
P1.3	IN4	Coil D

Schematic Description

- Port 1 pins of 8051 → ULN2003 inputs
- ULN2003 outputs → Stepper motor coils
- External power supply drives the motor
- ULN2003 protects 8051 from back EMF

(c) Stepper Motor Excitation Sequence

Clockwise Rotation

```
0001 → 0010 → 0100 → 1000
```

Anti-Clockwise Rotation

```
1000 → 0100 → 0010 → 0001
```

(d) 8051 Program for Speed and Direction Control

(Assembly Language)

Assumptions

- Stepper motor connected to **Port 1**
- Clockwise rotation with delay
- Speed controlled by changing delay value

PROGRAM

```
ORG 0000H

MAIN:
; CLOCKWISE ROTATION
    MOV A, #01H     ; Step 1
    MOV P1, A
    ACALL DELAY

    MOV A, #02H     ; Step 2
    MOV P1, A
    ACALL DELAY

    MOV A, #04H     ; Step 3
    MOV P1, A
    ACALL DELAY
```

```
    MOV A, #08H     ; Step 4
    MOV P1, A
    ACALL DELAY

    SJMP MAIN       ; Repeat rotation

; ---------------------
; Delay for speed control
; ---------------------
DELAY:
    MOV R0, #200    ; Change value to control speed
D1: MOV R1, #255
D2: DJNZ R1, D2
    DJNZ R0, D1
    RET

END
```

(e) Direction Control Explanation

- **Clockwise** → Forward excitation sequence
- **Anti-clockwise** → Reverse excitation sequence
- Direction can be changed by reversing the step pattern in program

(f) Speed Control Explanation

- Speed ∝ 1 / Delay
- **Increase delay → Motor speed decreases**
- **Decrease delay → Motor speed increases**
- Speed is controlled by changing delay loop count

(g) Output Observation

- Stepper motor rotates **step by step**
- Direction: **Clockwise**
- Speed: Controlled by delay subroutine
- Continuous rotation as long as program runs

(h) Applications

- Robotics
- CNC machines
- Printers & scanners
- Disk drives
- Industrial automation

4.Describe the different modes of operation of Timer/Counter in 8051 with its associated registers

Introduction

The **8051 microcontroller** contains **two 16-bit Timer/Counter units**, namely **Timer 0** and **Timer 1**.
They are used for:

- Time delay generation
- Event counting
- Baud rate generation
- Frequency measurement

Each timer can operate in **four different modes**, selected using the **TMOD register**.

TIMER vs COUNTER

- **Timer** → counts internal clock pulses
- **Counter** → counts external pulses at pins **T0 (P3.4)** and **T1 (P3.5)**

ASSOCIATED REGISTERS OF TIMER / COUNTER

1. TMOD – Timer Mode Register

Used to select:

- Timer or Counter
- Mode of operation

Bit	Function
GATE	Timer enable control
C/T	Timer (0) or Counter (1)
M1, M0	Mode selection

TMOD format:

```
| GATE | C/T | M1 | M0 | GATE | C/T | M1 | M0 |
|----- Timer 1 -----|----- Timer 0 -----|
```

---

2. TCON – Timer Control Register

Used to:

- Start/stop timers
- Indicate overflow

Bit	Function
TF1	Timer 1 overflow flag
TR1	Timer 1 run control
TF0	Timer 0 overflow flag
TR0	Timer 0 run control

---

3. TH0, TL0 / TH1, TL1

- **THx** → High byte of timer
- **TLx** → Low byte of timer
- Together form a **16-bit timer register**

MODES OF OPERATION

MODE 0 – 13-BIT TIMER MODE

Description

- Timer works as **13-bit timer**
- TLx uses 5 bits
- THx uses 8 bits
- Maximum count = **8192 (2^{13})**

Features

- Old mode (kept for compatibility)
- Rarely used in practice

Registers Used

- TMOD
- TH0 / TL0 or TH1 / TL1

- TCON

MODE 1 – 16-BIT TIMER MODE

Description

- Timer works as a **16-bit timer**
- Uses **THx and TLx fully**
- Count range: **0000H to FFFFH**

Features

- Most commonly used mode
- Suitable for accurate delays

Operation

- Timer increments every machine cycle
- On overflow, **TFx flag is set**

Registers Used

- TMOD
- THx, TLx
- TCON

MODE 2 – 8-BIT AUTO-RELOAD MODE

Description

- TLx works as **8-bit timer**
- THx holds **reload value**
- After overflow, TLx is automatically reloaded from THx

Features

- Automatic reload
- Widely used for **baud rate generation**

Advantages

- No need to reload timer manually
- Consistent time intervals

Registers Used

- TMOD
- THx (reload value)
- TLx
- TCON

MODE 3 – SPLIT TIMER MODE (TIMER 0 ONLY)

Description

- **Timer 0 is split** into two independent 8-bit timers
  - TL0 → acts as Timer 0
  - TH0 → acts as Timer 1
- Timer 1 becomes unavailable

Features

- Used when two 8-bit timers are required
- Rarely used

Registers Used

- TMOD
- TH0, TL0
- TCON

SUMMARY OF TIMER MODES

Mode	Size	Description	Usage
Mode 0	13-bit	Old timer mode	Rare
Mode 1	16-bit	Full timer	Delay generation
Mode 2	8-bit	Auto reload	Baud rate
Mode 3	Split	Two 8-bit timers	Special cases

TIMER OPERATION STEPS (COMMON)

1. Select mode using **TMOD**

2. Load initial value into **THx / TLx**
3. Start timer using **TRx**
4. Monitor overflow flag **TFx**
5. Stop timer after overflow

5 .INTERRUPTS IN 8051 MICROCONTROLLER

Definition of Interrupt

An **interrupt** is an internal or external event that temporarily halts the normal execution of the microcontroller to inform it that a device requires immediate service.
When an interrupt signal is received, the microcontroller suspends its current task and executes a special program known as the **Interrupt Service Routine (ISR)**.
After servicing the interrupt, the microcontroller resumes the execution of the main program.
Multiple devices can be serviced based on **interrupt priority**.

Execution of an Interrupt

The sequence of operations during interrupt handling is as follows:

1. The microcontroller completes the execution of the current instruction.
2. The **Program Counter (PC)** containing the address of the next instruction is saved on the stack.
3. The current interrupt status is internally saved.
4. The microcontroller jumps to a fixed memory location called the **interrupt vector address**.
5. The ISR located at that address is executed.
6. The ISR ends with a **RETI (Return from Interrupt)** instruction.
7. On executing RETI, the PC is restored from the stack and program execution resumes from where it was interrupted.

Interrupt Structure in 8051

The 8051 microcontroller supports **five interrupts**:

- **Three internal interrupts**
- **Two external interrupts**

Internal Interrupts

- Timer 0 overflow interrupt (TF0)
- Timer 1 overflow interrupt (TF1)
- Serial port interrupt (RI or TI)

External Interrupts

- **INT0** – External interrupt 0 (Pin P3.2)
- **INT1** – External interrupt 1 (Pin P3.3)

Types of Interrupts and Vector Addresses

Interrupt Source	Type	Vector Address
External Interrupt 0 (INT0)	External	0003H
Timer 0 Overflow (TF0)	Internal	000BH
External Interrupt 1 (INT1)	External	0013H
Timer 1 Overflow (TF1)	Internal	001BH
Serial Port (RI / TI)	Internal	0023H

Timer Flag Interrupts

- When **Timer 0 or Timer 1 overflows**, the corresponding flag **TF0 or TF1** is set to 1.
- This generates an interrupt and transfers control to the respective ISR.
- The flag is cleared automatically when the interrupt is serviced.

External Interrupts (INT0 and INT1)

- Located on pins **P3.2 (INT0)** and **P3.3 (INT1)**.
- Can be configured as:
  - **Level-triggered**
  - **Edge-triggered**
- Controlled using **TCON register**.
- Interrupt flags **IE0 and IE1** are set when the interrupt condition occurs.

Interrupt Control Registers

1. Interrupt Enable Register (IE)

The IE register enables or disables interrupts.

- **EA (D7)** must be set to 1 to allow interrupts.
- If EA = 0, all interrupts are disabled.

IE Register Bit Functions

Bit	Name	Function
D7	EA	Global interrupt enable
D4	ES	Enable serial interrupt
D3	ET1	Enable Timer 1 interrupt
D2	EX1	Enable External interrupt 1
D1	ET0	Enable Timer 0 interrupt
D0	EX0	Enable External interrupt 0

---

2. Interrupt Priority Register (IP)

- Determines priority of interrupts.
- Bit = 1 → High priority
- Bit = 0 → Low priority
- A high-priority interrupt can interrupt a low-priority ISR.

Priority Order (If Same Priority)

1. External Interrupt 0 (INT0)
2. Timer 0 interrupt
3. External Interrupt 1 (INT1)
4. Timer 1 interrupt
5. Serial port interrupt

---

3. Timer Control Register (TCON)

- Controls timer operation and external interrupts.
- Contains flags TF0, TF1, IE0, IE1 and control bits TR0, TR1.

Advantages of Interrupts

- Faster response to external events
- Efficient CPU utilization
- Eliminates continuous polling
- Essential for real-time systems

Introduction

Serial communication is the process of transferring data **one bit at a time** over a single communication line.
The **8051 microcontroller** has an **on-chip serial port (UART)** that supports **full-duplex serial communication**, allowing simultaneous transmission and reception of data.

Serial Port Pins in 8051

The serial communication in 8051 uses **Port 3 pins**:

- **RXD (P3.0)** – Receive data
- **TXD (P3.1)** – Transmit data

Serial Data Buffer Register (SBUF)

- SBUF is an **8-bit register**
- Used for **both transmission and reception**
- Writing data to SBUF → starts **data transmission**
- Reading data from SBUF → retrieves **received data**

Serial Control Register (SCON)

SCON is used to **configure and control serial communication**.

SCON Register Format

Bit	Name	Function
SM0	Serial mode select	
SM1	Serial mode select	
SM2	Multiprocessor communication	
REN	Receive enable	
TB8	9th transmit bit	
RB8	9th receive bit	
TI	Transmit interrupt flag	

Bit	Name	Function
RI	Receive interrupt flag	

Modes of Serial Communication in 8051

The 8051 supports **four serial modes**:

Mode	Description	Baud Rate
Mode 0	Shift register mode	Fixed
Mode 1	8-bit UART	Variable
Mode 2	9-bit UART	Fixed
Mode 3	9-bit UART	Variable

👉 **Mode 1** is the most commonly used mode.

Baud Rate Generation

- Baud rate is generated using **Timer 1**
- Controlled by **TMOD, TH1, and PCON registers**
- Common baud rates:
  - 4800 bps
  - 9600 bps

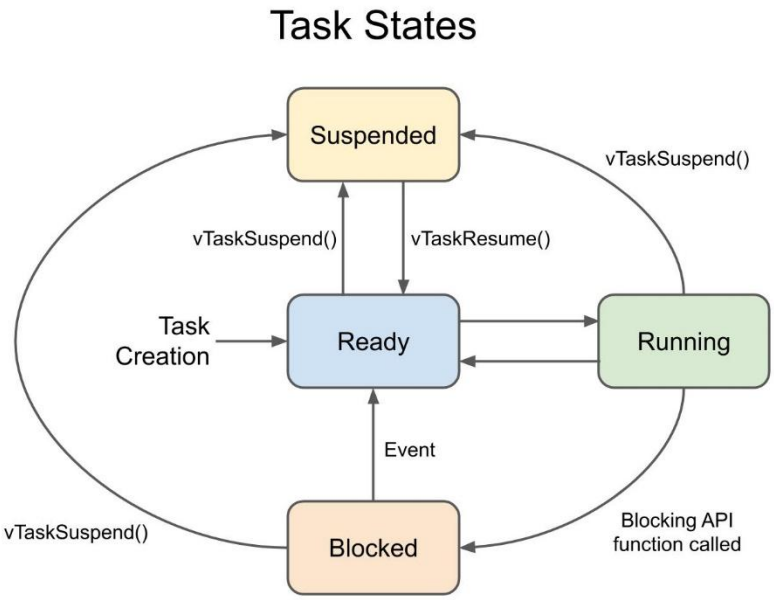
Serial Port Interrupt

- Serial communication generates an interrupt when:
  - **RI = 1** → data received
  - **TI = 1** → data transmitted
- RI and TI are **ORed** to generate a single serial interrupt
- Enabled using **ES bit in IE register**
- Interrupt vector address: **0023H**

Working of Serial Communication

1. Serial mode is selected using **SCON**
2. Baud rate is set using **Timer 1**
3. Data is loaded into **SBUF**
4. Bits are transmitted through **TXD**
5. On completion, **TI flag is set**
6. Received data is stored in SBUF and **RI flag is set**

7. VARIOUS STATES OF A TASK IN REAL-TIME OPERATING SYSTEMS (RTOS)



Introduction

In a **Real-Time Operating System (RTOS)**, a **task** (or process/thread) is the basic unit of execution.
A task does not execute continuously; instead, it moves through **different states** depending on CPU availability, interrupts, I/O events, and scheduling policies.
Understanding task states is essential for **real-time and deterministic system behavior**.

Various States of a Task in RTOS

---

1. New (Created) State

- The task is **created by the system**
- Task Control Block (TCB) is initialized
- Stack memory and priority are assigned

- Task has **not yet entered execution**

📌 This is a **temporary state**

---

2. Ready State

- Task is **ready to execute**
- All required resources are available
- Waiting for CPU allocation
- Task is placed in the **Ready Queue**

📌 If multiple tasks are ready, the **scheduler selects the highest-priority task**

---

3. Running State

- Task is **currently executing on the CPU**
- Only **one task can be in running state** on a single CPU
- Task continues execution until:
  - It completes
  - It is pre-empted
  - It waits for an event or I/O

---

4. Blocked (Waiting) State

- Task **cannot execute** because it is waiting for:
  - I/O operation
  - Semaphore / mutex
  - Message or signal
  - Interrupt or event
- CPU is **not wasted** on blocked tasks

📌 When the event occurs, task moves to **Ready state**

---

5. Delayed (Sleep) State

- Task is delayed for a **specific time interval**
- Used for **periodic tasks**
- Task remains inactive until delay expires

📌 After delay completion → task moves to **Ready state**

---

6. Suspended State

- Task is **intentionally suspended**
- Does not respond to events or scheduler
- Can be resumed only by:
  - Another task
  - An ISR (Interrupt Service Routine)

📌 Used for **control and debugging**

---

7. Terminated (Deleted) State

- Task has completed execution or is explicitly deleted
- Task resources are released
- Task is removed permanently from the scheduler

📌 This is the **final state**

Task State Transitions

From State	To State	Reason
New	Ready	Task admitted to scheduler
Ready	Running	CPU assigned
Running	Ready	Pre-emption by higher priority task
Running	Blocked	Waiting for event / I/O
Blocked	Ready	Event occurs
Running	Delayed	Delay function called
Delayed	Ready	Delay time expires
Running	Terminated	Task execution completed

Importance of Task States in RTOS

- Ensures **predictable timing behavior**
- Supports **priority-based pre-emptive scheduling**
- Improves **CPU utilization**
- Avoids busy waiting
- Helps meet **real-time deadlines**